简化版AI文本聊天程序

简化版AI文本聊天程序

- 例程效果
- 代码讲解
  - 1. 导入必要的模块
  - 2. 定义全局变量和常量
  - 3. 辅助函数
    - 3.1 WiFi连接函数
    - 3.2 绘制圆角矩形
    - 3.3 文本分页
    - 3.4 绘制文本（考虑间距）
    - 3.5 绘制加载动画
    - 3.6 绘制主界面
  - 4. 发送消息给AI
  - 5. 主程序逻辑
    - 5.1 主循环：状态管理和用户交互
    - 5.2 更新显示
    - 5.3 异常处理与资源释放
  - 6. 主程序入口

例程效果

本节例程源码文件名为 `simple_ai_chat.py`。

连接IDE，填写代码中的API_KEY、WIFI名称和WIFI密码，在IDE中运行代码，你将看到一个简化的AI文本聊天界面。程序会自动连接WiFi网络，并向AI发送一个预设问题（例如“给我讲个笑话吧”）。AI的回复将以多页形式显示，如果回复内容较长，会自动翻页。



代码讲解

本节将详细解析 `simple_ai_chat.py` 代码的功能和实现逻辑，帮助用户理解如何实现一个简化的AI文本聊天程序，包括WiFi连接、API请求、文本分页显示和用户交互。

1. 导入必要的模块

```
import uos
import time
import gc
import ujson as json
from media.display import *
from media.media import *
import ybUtils.YbKey as YbKey
import YbRequests as requests
```

- `uos`：用于操作系统相关操作，如退出点控制。
- `time`：用于时间控制。
- `gc`：用于垃圾回收，管理内存。
- `ujson`：用于处理JSON数据（别名为 `json`）。
- `media.display` 和 `media.media`：用于屏幕显示和媒体资源管理。
- `ybUtils.YbKey`：用于按键检测。
- `YbRequests`：用于发送HTTP网络请求。

2. 定义全局变量和常量

```
USER_QUESTION = "给我讲个笑话吧"
API_KEY = ""
WIFI_SSID = ""
WIFI_KEY = ""
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480
network_connected = False
```

- `USER_QUESTION`：预设的用户问题，将发送给AI。
- `API_KEY`，`WIFI_SSID`，`WIFI_KEY`：API密钥和WiFi连接信息（需用户填写）。
- `DISPLAY_WIDTH`，`DISPLAY_HEIGHT`：屏幕分辨率常量，分别为640x480。
- `network_connected`：布尔值，记录网络连接状态。

此外，代码还定义了界面颜色（`UI_COLORS`）、聊天历史记录（`chat_history`）、AI回复文本（`ai_response_text`）、分页信息（`response_pages`，`current_page`，`total_pages`）和程序状态（`program_state`）等变量。

3. 辅助函数

3.1 WiFi连接函数

```
def connect_wifi():
    """连接WiFi网络"""
    global network_connected
    try:
        import network
        wlan = network.WLAN(network.STA_IF)
        wlan.active(True)
        if not wlan.isconnected():
            wlan.connect(WIFI_SSID, WIFI_KEY)
            timeout = 10
            while not wlan.isconnected() and timeout > 0:
```

```
                time.sleep(1)
                timeout -= 1
            if wlan.isconnected():
                network_connected = True
                return True
            else:
                return False
        else:
            network_connected = True
            return True
    except Exception as e:
        print(f"WiFi连接失败: {e}")
        return False
```

- `connect_wifi()`：尝试连接WiFi网络，设置10秒超时。如果连接成功，设置`network_connected`为`True`并返回`True`；否则返回`False`。

3.2 绘制圆角矩形

```
def draw_rounded_rect(img, x, y, width, height, color, corner_radius=12,
fill=True):
```

- `draw_rounded_rect()`：在图像上绘制圆角矩形，通过组合矩形和圆形实现圆角效果。

3.3 文本分页

```
def split_text_to_pages(text, max_chars_per_page=400):
```

- `split_text_to_pages()`：将长文本按指定字符数（默认400）分页，以句号为分隔符，确保每页内容不超过限制。

3.4 绘制文本（考虑间距）

```
def draw_text_with_proper_spacing(img, x, y, width, height, text, font_size=16,
color=(28, 28, 30)):
```

- `draw_text_with_proper_spacing()`：在指定区域内绘制文本，自动换行并考虑中英文字符宽度差异，确保文本显示整齐。

3.5 绘制加载动画

```
def draw_loading_spinner(img, x, y, frame):
```

- `draw_loading_spinner()`：绘制一个旋转的加载动画，用于表示AI正在思考或网络请求中。

3.6 绘制主界面

```
def draw_main_interface(img, loading_frame=None):
```

- `draw_main_interface()`：绘制整个聊天界面，包括状态栏、问题区域、AI回复区域和底部状态信息。支持显示加载动画、分页信息和重试进度条。

4. 发送消息给AI

```
def send_message_to_ai(message):
    global chat_history, ai_response_text, response_pages, total_pages,
current_retry
```

- `send_message_to_ai()`：向AI发送消息并获取回复。
- 检查网络连接状态，如果未连接则返回错误信息。
- 实现重试逻辑（最多3次），发送HTTP POST请求到指定的API端点（阿里云DashScope）。
- 处理API响应，成功时更新聊天历史和分页显示内容；失败时根据错误类型决定是否重试，并更新错误信息。
- 返回请求是否成功及回复或错误信息。

5. 主程序逻辑

```
def main():
    global current_page, program_state
    key = YbKey.YbKey()
    Display.init(Display.ST7701, width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT,
to_ide=True)
    MediaManager.init()
    img = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.ARGB8888)
```

- 初始化按键检测、显示器和媒体管理器。
- 创建一个ARGB8888格式的图像对象 `img`，用于界面绘制。

5.1 主循环：状态管理和用户交互

```
while True:
    uos.exitpoint()
    current_time = time.ticks_ms()
    key_pressed = key.is_pressed() == 1
```

- 持续检测按键状态：
  - 如果检测到长按（超过2秒），重新连接WiFi并更新程序状态。
- 根据程序状态（`program_state`）执行不同操作：
  - `connecting`：尝试连接WiFi，成功后进入 `asking` 状态。
  - `asking`：等待2秒后进入 `responding` 状态，发送问题给AI。
  - `responding`：显示加载动画，等待AI回复。
  - `displaying`：显示AI回复内容，多页时每4秒自动翻页。
  - `completed`：对话完成，停止翻页。

5.2 更新显示

```
if loading_frame is not None:
    if current_time - loading_start_time > 100:
        loading_frame += 1
        loading_start_time = current_time
draw_main_interface(img, loading_frame)
Display.show_image(img)
```

- 更新加载动画帧（每100毫秒）。

- 绘制主界面并显示在屏幕上。
- 每5秒执行一次垃圾回收（`gc.collect()`），优化内存使用。

5.3 异常处理与资源释放

```
except KeyboardInterrupt:
    print("用户停止程序")
except Exception as e:
    print(f"程序异常: {e}")
finally:
    Display.deinit()
    uos.exitpoint(uos.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
    MediaManager.deinit()
    print("程序已退出")
```

- 捕获用户中断（`KeyboardInterrupt`）和其他异常（`Exception`），并打印相关信息。
- 反初始化显示器，设置退出点为睡眠模式，并释放媒体资源。

6. 主程序入口

```
if __name__ == "__main__":
    main()
```

- 调用 `main()` 函数启动程序。